

Planeja Mobile

Plano de Desenvolvimento

Roteiro passo a passo para **Maria, Mario e Ana** começarem juntos, no mesmo dia, e chegarem ao encerramento em **17 de dezembro de 2026**.

Disciplina: Desenvolvimento de Dispositivos Móveis · 6º semestre — Sistemas de Informação.

1. Contexto

O Planeja migra da web para mobile (React Native + Expo). O escopo descreve 6 CRUDs (3 herdados: destinos, roteiros e atividades; 3 novos: dicas, ranking e grupos/despesas), 25 requisitos (RF-01–RF-25) e uma divisão de tarefas por pessoa.

Descoberta que muda o plano (a favor de vocês): o backend **já está construído**. A pasta `server/` contém a API completa — todos os 11 modelos, todas as migrations (inclusive as 7 tabelas novas) e as rotas/controllers/services de auth, destinos, roteiros, atividades, dicas, avaliações/ranking, grupos e despesas, incluindo o cálculo de acerto de contas (`GET /api/grupos/:id/contas`).

Ou seja: o "estender a API" do escopo está ~90% pronto. **O trabalho real é o app mobile** (consumir a API já existente) + publicar a API para a apresentação. Isso é ótimo para o prazo curto (Set → Dez).

Correções em relação ao texto do escopo (o PDF está desatualizado)

- **Banco:** o PDF diz MySQL; o código real é PostgreSQL no Supabase (`pg` , `DATABASE_URL` , migration de RLS).
- **Rotas novas:** o PDF lista como "a fazer"; já existem em `server/src/routes/` .

Decisões travadas nesta reunião

- **Expo SDK 54** (conforme AGENTS.md, para o Expo Go da loja abrir o QR). O template do professor está em SDK 57 — vamos reproduzir a estrutura dele, mas fixar as dependências nas versões do SDK 54.
- **Fotos das dicas → Supabase Storage.** O app envia a imagem para um bucket, recebe a URL pública e manda essa URL para o endpoint que já existe (`POST /api/dicas/:id/fotos`). Sem alterar a API.

2. Stack e padrões a reproduzir do template do professor

Base de referência: `app-moretto/` . Reproduzir **estrutura e padrões**, não o conteúdo.

Camada	Padrão a copiar	Referência
Estrutura de pastas	src/{components,screens,services,theme,constants}	app-moretto/src/
Atomic Design	components/{atoms,molecules,organisms,templates}	Button.js, ScreenTemplate.js
Cliente HTTP	helper apiRequest com Bearer + tratamento de erro	services/api.js
URL base por plataforma	Platform.select + variável de ambiente	services/config.js
Sessão no device	expo-secure-store (web cai em localStorage)	services/storage.js
Tokens visuais	cores/espacamentos/raios/tipografia centralizados	theme/index.js

Duas diferenças a resolver na base (o template não tem):

1. **Navegação por abas.** O template só usa native-stack. O escopo pede 5 abas inferiores (Início, Roteiros, Dicas, Ranking, Grupos). Adicionar `@react-navigation/bottom-tabs`.
2. **Envelope de resposta da API.** O backend responde `{ data: ... }` no sucesso e `{ error: { message, details } }` no erro. Adaptar o apiRequest para ler `data?.error?.message`, e as telas usam `resposta.data`.

Paleta Planeja (cap. 09) para o theme, no lugar do azul do template: Acento #8A3A1F · Tinta #1A1A1A · Texto suave #6B6B6B · Papel #FAF8F3 · Linha #D8D4CC. Tipografia: Fraunces (títulos) + Source Sans 3 (texto).

3. Divisão de tarefas (cap. 13)

Cada pessoa é dona de uma **fatia vertical** inteira: serviço de API → telas → validação → teste.

Pessoa	Frente compartilhada	CRUD herdado	CRUD novo
Maria	Arquitetura, navegação, contexto de auth, sessão	Roteiros (RF-06)	Grupos + Despesas (RF-17–RF-25)
Mario	API/infra, deploy, bucket Storage, PRs/builds	Destinos (RF-05)	Ranking (RF-12–RF-16)
Ana	Componentes visuais/identidade, upload de imagem, ClickUp/doc	Atividades (RF-07)	Dicas + fotos (RF-08–RF-11)

Auth (login/cadastro/esqueci-senha/perfil) e a tela Início são da Maria (frente compartilhada), mas construídas na Fase 1 com todo mundo junto.

4. O método da "fatia vertical" (como cada CRUD é feito)

Todo CRUD segue os mesmos 6 passos, na ordem. É também a granularidade das tarefas no ClickUp.

1. **Service** — arquivo em `src/services/<recurso>.js` com listar/criar/buscarPorId/atualizar/excluir, reusando o apiRequest.
2. **Tela de listagem** — FlatList com estados de carregando/vazio/erro (RNF-06/07).
3. **Formulário** — criar/editar, usando os componentes de molecules/FormField.
4. **Validação no app** — antes de enviar (espelhando as regras do validator da API).

5. **Ligar à navegação** — botões, params de rota, voltar após salvar.
6. **Testar no aparelho físico** (RNF-01) — fluxo completo: cria, edita, exclui, com token real.

Regra de ouro: ninguém abre a fatia dele antes da base compartilhada (Fase 1) estar de pé. Sem `apiRequest` + `auth` + `navegação`, cada CRUD traria no login.

5. Cronograma calibrado (Set 24 → Dez 17 ≈ 12 semanas)

O escopo previa 17 semanas; recalibrado para as semanas reais restantes.

Fase 0 — Preparação (esta semana, ~3 dias) · TODOS

Objetivo: todo mundo com o backend rodando e um "hello world" no próprio celular.

- ❑ **Mario:** criar projeto no Supabase, pegar `DATABASE_URL`, subir o banco (`cd server && npm install` , copiar `.env`, rodar `npm run db:reset`). Validar `GET /health` .
- ❑ **Mario:** criar o bucket público de imagens no Supabase Storage (ex.: `dicas`).
- ❑ **Todos:** instalar Node, Expo Go (SDK 54) no celular, VS Code, Git.
- ❑ **Maria:** criar o repositório `mariahoppe/app-planeja` com a main protegida.
- ❑ **Ana:** montar o board no ClickUp (uma lista por frente). Sprints de 2 semanas.

Entregável: backend no Supabase respondendo; todos rodam Expo Go; board pronto.

Fase 1 — Base compartilhada + Autenticação (semanas 1–2) · TODOS juntos

A parte que destrava o trabalho paralelo. Fazer em par/mob, num branch feat/base.

- ❑ `create-expo-app` fixado em SDK 54; recriar a árvore `src/` do template.
- ❑ `theme/index.js` com a paleta Planeja + fontes (Fraunces/Source Sans 3).
- ❑ `services:` `config.js` (`URL base` + `EXPO_PUBLIC_API_URL`), `api.js` (`envelope { data }/{ error }`) e `storage.js`.
- ❑ `AuthContext` (`obterUsuario` na abertura → rota inicial) + sessão persistida.
- ❑ Navegação: stack de Auth (Login, Cadastro, EsqueciSenha) + `BottomTabs` das 5 seções, cada aba com seu stack.
- ❑ Telas de conta: Login, Cadastro, Esqueci a senha, Perfil (RF-01–RF-04).
- ❑ Componentes base compartilhados (Ana): `Button`, `Input`, `FormField`, `Card`, `ScreenTemplate`, estados vazios.

Entregável: login/perfil funcionando no celular; as 5 abas navegam (telas ainda vazias).

Fase 2 — CRUDs herdados em paralelo (semanas 3–6)

- ❑ **Mario** → **Destinos** (`/api/destinos`) + tela Início (`/api/dashboard`).
- ❑ **Maria** → **Roteiros** (`/api/roteiros` , inclui `/id/resumo` e vínculo com destino).
- ❑ **Ana** → **Atividades** (`/api/atividades` , com `PATCH /:id/toggle`).

Entregável: 3 CRUDs herdados navegáveis, criando/editando/excluindo dados reais.

Fase 3 — CRUDs novos (semanas 7–10)

- ❑ **Ana → Dicas + fotos** (`/api/dicas`): 4 blocos, busca por destino, expo-image-picker → upload ao Supabase Storage → `POST /:id/fotos`. Comprimir imagem (RNF-09).
- ❑ **Mario → Ranking** (`/api/avaliacoes/ranking?escopo=`): duas listas (países/cidades), filtro, e tocar no destino abre Dicas filtrada (RF-16).
- ❑ **Maria → Grupos + Despesas** (`/api/grupos*`): criar/entrar por código, aba de despesas, divisão igual/personalizada, e acerto de contas (`GET /:id/contas`).

Entregável: os 6 CRUDs completos e integrados (ranking → dicas → roteiro → grupo).

Fase 4 — Testes, deploy e entrega (semanas 11–12)

- ❑ **Mario:** publicar a API num host gratuito (Render/Railway/Fly); trocar EXPO_PUBLIC_API_URL.
- ❑ Testes em aparelhos diferentes (Android físico obrigatório, RNF-01); correção de bugs.
- ❑ Refino visual, áreas de toque ≥44pt (RNF-03), acessibilidade básica.
- ❑ **Build do APK** via EAS Build (`eas build -p android --profile preview`).
- ❑ Gravar vídeo demonstrativo + preparar apresentação.

Congelamento de escopo 2 semanas antes: só correções. **Entregável:** APK gerado, vídeo, apresentação — 17 dez.

6. Referência rápida da API

Base: `http://<IP-da-maquina>:3000/api` (dev) · header `Authorization: Bearer <token>` nas protegidas.
Sucesso: `{ data }` · Erro: `{ error: { message } }`.

Frete	Endpoints principais
Conta	<code>POST /auth/registro</code> · <code>/auth/login</code> · <code>/auth/esqueci-senha</code> · <code>/auth/redefinir-senha</code> · <code>GET /usuarios/perfil</code> · <code>PUT /usuarios/trocar-senha</code>
Início	<code>GET /dashboard</code>
Destinos	<code>GET/POST /destinos</code> · <code>GET/PUT/DELETE /destinos/:id</code>
Roteiros	<code>GET/POST /roteiros</code> · <code>GET/PUT/DELETE /roteiros/:id</code> · <code>GET /roteiros/:id/resumo</code>
Atividades	<code>GET/POST /atividades</code> · <code>GET/PUT/DELETE /atividades/:id</code> · <code>PATCH /atividades/:id/toggle</code>
Dicas	<code>GET /dicas</code> · <code>GET /dicas/minhas</code> · <code>GET/POST /dicas</code> · <code>PUT/DELETE /dicas/:id</code> · <code>POST /dicas/:id/fotos</code> · <code>DELETE /dicas/:id/fotos/:fotold</code>
Ranking	<code>GET /avaliacoes/ranking?escopo=</code> · <code>GET /avaliacoes/minhas</code> · <code>POST /avaliacoes</code> · <code>DELETE /avaliacoes/:id</code>
Grupos	<code>GET/POST /grupos</code> · <code>POST /grupos/entrar</code> · <code>GET/PUT/DELETE /grupos/:id</code> · <code>DELETE /grupos/:id/integrantes/:usuariold</code> · <code>POST /grupos/:id/sair</code>
Despesas	<code>GET /grupos/:id/contas</code> · <code>GET/POST /grupos/:id/despesas</code> · <code>GET/PUT/DELETE /grupos/:id/despesas/:despesald</code>

Contas de teste do seed: `maria@planeja.app` / `mario@planeja.app` / `ana@planeja.app` — senha `planeja123`.

7. Fluxo de Git e ClickUp (RNF-11)

- **Branch por tarefa:** `feat/dicas-publicar`, `fix/login-token`. Ninguém escreve na main.
- **Commits convencionais:** `feat`, `fix`, `refactor`, `docs`, `style`.
- **PR para main** descrevendo o que muda e como testar, com link da tarefa no ClickUp.
- **Revisão:** aprovação de pelo menos 1 integrante que não escreveu o código, antes do merge.
- `.env` fora do versionamento; commitar apenas `.env.example`.
- **ClickUp:** A fazer → Em desenvolvimento → Em revisão → Concluído (passa a "Em revisão" ao abrir o PR). Cada tarefa cita o RF que atende.

8. Rede em desenvolvimento (o erro nº 1 de todo mundo)

O celular não enxerga o localhost do PC. Em `services/config.js`:

- **Android emulador:** `http://10.0.2.2:3000`
- **iOS simulador:** `http://localhost:3000`
- **Aparelho físico (Expo Go):** `http://<IP-da-máquina-na-LAN>:3000` — descubra com `ipconfig`.

Deixar a URL em `EXPO_PUBLIC_API_URL` para alternar dev/produção sem mudar código.

9. Verificação (como saber que está funcionando)

1. **Backend:** `npm run dev` → `/health` OK; login com `maria@planeja.app/planeja123` retorna `{ data: { token } }`.
2. **App na base (Fase 1):** abrir no Expo Go, logar, ver perfil, navegar entre as 5 abas, fechar e reabrir sem logar de novo (RNF-05).
3. **Cada CRUD (Fases 2–3):** no aparelho físico, criar → lista → editar → excluir, com estados de carregando/vazio/erro.
4. **Fotos:** publicar dica com foto da galeria → imagem no Supabase Storage e na dica.
5. **Ranking → Dicas:** tocar num destino do ranking abre Dicas já filtrada por ele.
6. **Acerto de contas:** lançar 2–3 despesas → a lista "quem paga a quem" fecha com os totais.
7. **Entrega:** APK instala e roda num Android físico apontando para a API publicada.

10. Riscos e cuidados

- **Não quebrar a API web:** mudanças na API devem ser aditivas. Evite mexer nela além do bucket de Storage e do deploy.
- **SDK 54 fixo:** ao instalar libs, usar `npm install <lib>` (versão compatível) — nunca `npm install cru`.

- **Upload de foto** é a tarefa mais nova/arriscada (Ana): prototipar cedo, isolada.
 - **Deploy da API** perto da entrega dá sustos: Mario faz um deploy "de mentira" já na Fase 2.
 - **JWT:** .env.example usa 7d; o README fala 24h. Confirmar JWT_EXPIRES_IN e testar expiração.
-

Documento gerado a partir do plano aprovado. Editável — ajustem à vontade na primeira reunião de sprint.